TypeScript Handbook

TypeScript Notes

1. What is TypeScript ?

→ TypeScript is a superset of JavaScript.
→ It adds static typing, classes, interfaces, and other features to JavaScript.
→ TypeScript code is compiled to plain JavaScript.
→ It helps in writing better, safer and more maintainable code.
→ It is developed and maintained by Microsoft.

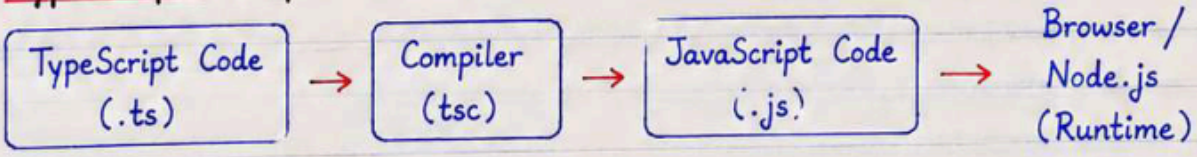
2. TypeScript vs JavaScript

JavaScript	TypeScript
→ Designed for web development.	→ Built for large scale applications.
→ Dynamically typed.	→ Statically typed.
→ No type checking.	→ Type checking during development.
→ Errors found at runtime.	→ Errors found at compile time.
→ Less code maintainability.	→ More code maintainability.
→ No interfaces or types.	→ Supports interfaces, types, enums.
→ Directly runs in browser.	→ Needs compilation before running.

3. Static vs Dynamic Typing

→ JavaScript is dynamically typed.
→ TypeScript is statically typed.
→ In static typing, type is known at compile time.
→ In dynamic typing, type is known at runtime.

4. TypeScript Compilation Flow



5. File Extensions

→ .ts → TypeScript file
→ .js → JavaScript file

2. TypeScript Types & Type Annotations

→ Primitive Types

1. string → represents text.	`let name: string = "Aman";`
2. number → represents numbers.	`let age: number = 25;`
3. boolean → true or false.	`let isLoggedIn: boolean = true;`
4. null → intentional absence of any value.	`let value: null = null;`
5. undefined → a variable declared but not assigned.	`let data: undefined = undefined;`
6. any → can hold any type of value.	`let anything: any = 10;` `anything = "hello";`
7. unknown → type safe alternative of any.	`let notSure: unknown = "test";` `// need type check before use`

8. void → used for functions that do not return anything.	`function log(): void {`
	`  console.log("Hello");`
	`}`

9. never → represents values that never occur.	`function throwError(msg: string): never {`
	`  throw new Error(msg);`
	`}`

→ Type Inference

TypeScript automatically infers the type based on the value.

```
let city = "Delhi";   // inferred as string        let count = 100;  // inferred as number
```

→ Type Annotations

We can explicitly specify the type.

```
let country: string = "India";                    let population: number = 140;
```

→ Type Assertions

We can change or tell the compiler the type using assertions.

```
let someValue: any = "Hello TypeScript";
let strLength: number = (someValue as string).length;
let empId = <number>someValue;     // another way
```

3. Arrays, Tuples, Objects & Enums

① Arrays

→ An array is a collection of elements of the same type.
→ We can define arrays using type[] or Array<type>.

Example:

```
let numbers: number[] = [1, 2, 3, 4];
let names: Array<string> = ["Aman", "Neha"];
let fruits: string[] = ["Apple", "Banana"];
```

② Readonly Arrays

→ Once created, we cannot change the array.
→ Useful when we want to prevent modifications.

Example:

```
let ids: readonly number[] = [101, 102, 103];
// ids.push(104);     // Error
// ids[0] = 200;      // Error
```

③ Tuples

→ Tuples allow us to store a fixed number of elements of different types.
→ Order and type are important.

Example:

```
let user: [number, string, boolean];
user = [1, "Aman", true];
// user = ["Aman", 1, true]; // Error
```

④ Named Tuples

→ Tuples with names for each element.
→ Improves readability.

Example:

```
let user: [id: number, name: string, isActive: boolean];
user = [1, "Aman", true];
console.log(user.name);   // Aman
```

⑤ Objects

→ Used to define complex data.
→ Key-value pairs.
→ Keys are strings (or number/symbol).

Example:

```
let person: { name: string; age: number; isActive: boolean };
person = { name: "Aman", age: 25, isActive: true };
```

⑥ Optional Properties

→ Some properties may not be present.
→ Use ? to mark optional.

Example:

```
let product: { id: number; name: string; price?: number };
product = { id: 1, name: "Laptop" };  // price optional
```

⑦ Readonly Properties

→ Once initialized, value cannot be changed.
→ Use readonly keyword.

Example:

```
let config: { readonly apiUrl: string; timeout: number };
config = { apiUrl: "https://api.com", timeout: 5000 };
// config.apiUrl = "https://test.com"; // Error
```

⑧ Enums

→ Enums allow us to define a set of named constants.
→ Improves readability and maintainability.

Example:

```
enum Status {
    Pending = 1,
    Success = 2,
    Failed  = 3
}
let currentStatus: Status = Status.Pending;
```

4. Functions in TypeScript

1. Basic Function with Types

→ We can specify types for parameters and return value.
→ TypeScript checks types at compile time.

Example:

```
function add(a: number, b: number): number {
    return a + b;
}
let result: number = add(5, 10);  // result = 15
```

2. Optional Parameters

→ We can mark parameters as optional using ?.
→ Optional parameters must come after required ones.

Example:

```
function greet(name: string, greeting?: string): string {
    return greeting ? greeting + "," + name : "Hello, " + name;
}
greet("Aman");          // Hello, Aman
greet("Aman", "Hi");    // Hi, Aman
```

3. Default Parameters

→ We can provide default values for parameters.
→ If value is not passed, default value will be used.

Example:

```
function multiply(a: number, b: number = 1): number {
    return a * b;
}
multiply(5);        // 5
multiply(5, 3);     // 15
```

4. Rest Parameters

→ Rest parameters allow a function to accept multiple values.
→ Use ... before the parameter.

Example:

```
function sum(...nums: number[]): number {
    let total = 0;
    for (let n of nums) total += n;
    return total;
}
sum(1, 2, 3);              // 6
sum(10, 20, 30, 40);       // 100
```

5. Function Types

→ We can define the type of a function using type annotations.
→ Useful in callbacks and variables.

Example:

```
let operate: (a: number, b: number) => number;
operate = function(a: number, b: number): number {
    return a - b;
}
let ans = operate(10, 4);    // 6
```

6. Arrow Functions

→ Short syntax for writing functions.
→ Works same as normal functions.

Example:

```
let square = (n: number): number => {
    return n * n;
}
console.log(square(4));   // 16
```

7. Void vs Never

→ void: function returns nothing.
→ never: function never returns.

Example:

```
function logMsg(msg: string): void {
    console.log(msg);
}
function throwError(msg: string): never {
    throw new Error(msg);
}
```

5. Interfaces vs Type Aliases

1. Interface

→ Used to define the structure of an object.

→ Can be extended using extends.

→ Can be implemented by classes using implements.

→ Mostly used for object shapes.

Example:

```
interface Person {
    id: number;
    name: string;
    isActive?: boolean;    // optional property
}
let person: Person = {
    id: 1,
    name: "Aman",
    isActive: true
};
```

2. Type Alias

→ Used to give a new name to any type.

→ Can represent primitive, union, tuple, object, anything.

→ Cannot be implemented by classes.

→ More flexible than interface.

Example:

```
type ID = number;
type Name = string;

type User = {
    id: ID;
    name: Name;
    isActive: boolean;
};
let user: User = {
    id: 101,
    name: "Neha",
    isActive: false
};
```

3. Interface vs Type Alias (Comparison)

→ Interface is used for object shapes.

→ Type alias can be used for any type.

→ Interface supports declaration merging.

→ Type alias does not support merging.

→ Interface can be extended using extends.

→ Type alias can use intersection (&) for extending.

→ Interface can be implemented by classes.

→ Type alias cannot be implemented by classes.

Example:

```
// Interface (Extending)
interface Animal {
    name: string;
}
interface Dog extends Animal {
    breed: string;
}

// Type Alias (Extending using &)
type AnimalType = {
    name: string;
}
type DogType = AnimalType & {
    breed: string;
};
```

6. Classes & Objects in TypeScript

- → TypeScript supports class based object oriented programming.
- → We can create classes, objects, constructors, methods.
- → Access modifiers: public, private, protected.
- → Supports inheritance using extends.
- → Supports implements using interfaces.
- → Supports getters and setters.

Example:

```
class Person {
    public id: number;
    public name: string;

    constructor(id: number, name: string) {
        this.id = id;
        this.name = name;
    }

    getDetails(): string {
        return this.id + " - " + this.name;
    }
}
let p = new Person(1, "Aman");
console.log(p.getDetails());     // 1 - Aman
```

7. Inheritance

- → Allows a class to inherit properties and methods from another class.
- → Use extends keyword.
- → Promotes code reusability.
- → Supports method overriding.
- → Super keyword is used to call parent class constructor and methods.

Example:

```
class Animal {
    constructor(public name: string) { }
    speak(): void {
        console.log(this.name + " makes sound");
    }
}
class Dog extends Animal {
    constructor(name: string, public breed: string) {
        super(name);
    }
    speak(): void {
        console.log(this.name + " barks");
    }
}
let d = new Dog("Tommy", "Labrador");
d.speak();     // Tommy barks
```

8. Access Modifiers

- → public - accessible everywhere.
- → private - accessible only within class.
- → protected - accessible within class and child classes.
- → readonly - value cannot be changed after initialization.

Example:

```
class Employee {
  public id: number;
  private salary: number;
  protected department: string;
  readonly code: string;

  constructor(id: number, salary: number, dept: string, code: string) {
      this.id = id;
      this.salary = salary;
      this.department = dept;
      this.code = code;    // cannot be changed later
  }
}
```

9. Union, Intersection & Literal Types

1. Union Types

→ A variable can hold one value from multiple types.

→ Created using | (pipe) operator.

→ Useful when value can be one of several types.

Example:

```
let id: number | string;
id = 101;                    // number
id = "Aman";                 // string

function printId(id: number | string) {
    console.log(id);
}
```

2. Intersection Types

→ Combines multiple types into one.

→ Created using & (ampersand) operator.

→ Result type has all properties from both types.

Example:

```
type Employee = {
    id: number;
    name: string;
};
type Manager = {
    department: string;
};
type ManagerEmployee = Employee & Manager;
let emp: ManagerEmployee = {
    id: 1,
    name: "Aman",
    department: "IT"
};
```

3. Literal Types

→ Allow a variable to have a specific value.

→ Created using exact value (string or number).

→ Useful for restricting values.

Example:

```
let status: "pending" | "success" | "failed";
status = "pending";          // valid
status = "success";          // valid
// status = "done";          // Error

type Direction = "up" | "down" | "left" | "right";
let move: Direction = "up";
```

4. Type Narrowing

→ Process of refining types to more specific types.

→ TypeScript understands the type based on conditions.

→ Common with union types.

Example:

```
function area(value: number | string) {
    if (typeof value === "number") {
        return value * value;
    } else {
        return value.length;
    }
}
console.log(area(5));        // 25
console.log(area("Hello"));  // 5
```

10. Generics in TypeScript

→ Generics provide a way to create reusable components.

→ Type can be passed as a parameter to functions, interfaces, classes.

→ Helps to maintain type safety.

→ Better than using any.

Example:

1. Generic Function

```
function getFirst<T>(items: T[]): T {
    return items[0];
}
let num = getFirst<number>([1, 3, 3, 4]);     // 1
let str = getFirst<string>(["a", "b", "c"]);  // a
```

2. Generic Interface

```
interface ApiResponse<T> {
    data: T;
    success: boolean;
    message: string;
}
let res: ApiResponse<string> = {
    data: "Hello",
    success: true,
    message: "OK"
};
```

11. Advanced Types & Utility Types

→ TypeScript provides built-in utility types to make development easier.

→ They help in transforming and modifying existing types.

→ Common utility types: Partial, Required, Readonly, Pick, Omit, Record, Exclude, Extract, ReturnType, Parameters.

Example:

```
interface User {
    id: number;
    name: string;
    email: string;
    isActive: boolean;
}
let updateUser: Partial<User>;                  // all properties optional
let createUser: Required<User>;                 // all properties required
let userEmail: Pick<User, 'id' | 'email'>;      // pick selected
let userWithoutEmail: Omit<User, 'email'>;      // omit email
let userMap: Record<string, User>;              // key-value map
let notBoolean: Exclude<string | number | boolean, boolean>;
let onlyString: Extract<string | number | boolean, string>;
type UserIdType = ReturnType<typeof getUserId>;
type ParamType = Parameters<typeof getUser>[0];
// These utility types save a lot of manual work.
```

12. TypeScript Configuration & Modules

→ tsconfig.json is the configuration file for TypeScript.

→ Controls compiler options.

→ Modules allow us to split code into multiple files.

→ Export and Import are used to share code.

Example:

```
// tsconfig.json (important options)
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "CommonJS",
    "strict": true,
    "rootDir": "src",
    "outDir": "dist",
    "esModuleInterop": true
  }
}
```

```
// Export (user.ts)
export interface User {
  id: number;
  name: string;
}
```

```
// Import (app.ts)
import { User } from './user";
let u: User = { id: 1, name: "Aman" };
```

13. Type Inference & Type Assertions

1. Type Inference

→ TypeScript automatically infers type based on value.

→ Helps reduce manual type annotations.

→ Works well in most common cases.

Example:

```
let num = 10;          // inferred as number
let str = "Aman";      // inferred as string
let isActive = true;   // inferred as boolean
let arr = [1, 2, 3];   // inferred as number[]
let user = { id: 1, name: "Neha" };
    // inferred as { id: number, name: string }
```

2. Type Assertions

→ Sometimes we know more about the type than TypeScript.

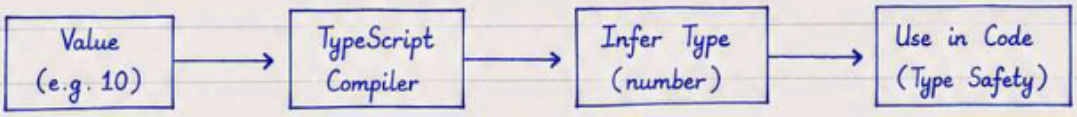
→ We can override the inferred type using assertions.

→ Uses "as" keyword or <Type> syntax.

Example:

```
let value: any = "Hello";
let len = (value as string).length;        // as syntax
let len2 = (<string>value).length;         // angle bracket syntax
let num = value as unknown as number;      // double assertion
```

Type Inference Flow



Summary: → Inference makes code cleaner.

→ Assertions are useful when working with any, DOM, or external libraries.

---

14. Enums in TypeScript

→ Enums allow us to define a set of named constants.

→ Improves readability and maintainability.

→ Can be numeric or string based.

→ Useful when values are fixed and known in advance.

Pro Tip:

Use enums when values are limited and won't change frequently.

Example:

1. Numeric Enum

```
enum Direction {
    Up, Down, Left, Right
}
let dir: Direction = Direction.Up;   // 0
console.log(Direction.Right);        // 3
```

```
Up    = 0
Down  = 1
Left  = 2
Right = 3
```

2. String Enum

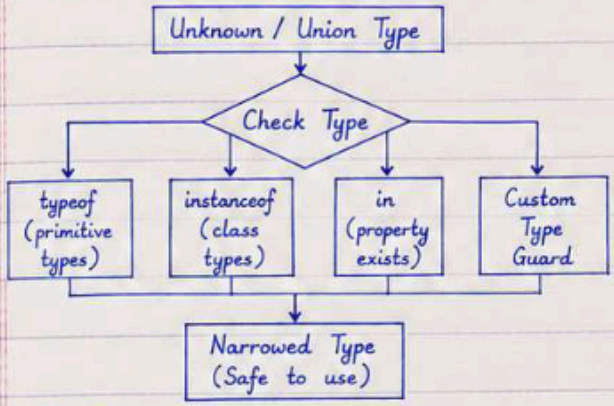
```
enum Status {
    Pending = "PENDING",
    Success = "SUCCESS",
    Failed  = "FAILED"
}
let s: Status = Status.Success;
console.log(s);      // SUCCESS
```

```
Pending = "PENDING"
Success = "SUCCESS"
Failed  = "FAILED"
```

15. Type Guards & Type Casting

- → Type guards help us narrow down types within a block.
- → Use typeof, instanceof, in, & custom type guards.
- → Type casting (type assertion) tells TypeScript to treat a value as a specific type.
- → Useful when we are sure about the type.

Type Guard Flow



Pro Tip:

Always prefer type guards over type assertions for safer code.

Example:

1. Using typeof

```
function print(value: string | number) {
    if (typeof value === "string") {
        console.log(value.toUpperCase());   // string methods
    } else {
        console.log(value.toFixed(2));      // number methods
    }
}
```

2. Using instanceof

```
class Car { start() { console.log("Car started"); } }
const v: Car | string = new Car();
if (v instanceof Car) {
    v.start();    // safe
} else {
    console.log(v.toUpperCase());
}
```

3. Type Assertion (Casting)

```
let someValue: any = "Hello";
let strLen = (someValue as string).length;   // 5
let num = <number>someValue;    // also works (not recommended)
```

16. Index Signatures & keyof Operator

- → Index signatures allow an object to have dynamic keys.
- → keyof returns union of all keys of an object type.
- → Useful for creating flexible and reusable types.
- → Used in dictionaries, maps, and utility types.

Common Index Signatures

Key Type	Syntax	Example
string	[key: string]: Type	{ name: string }
number	[index: number]: Type	[0: string]
string | number	[key: string | number]: Type	{ [key: string | number]: any }

Example:

1. Index Signature

```
interface Dictionary {
    [key: string]: string;
}
let dict: Dictionary = {
    hello: "Hi",
    bye: "Goodbye"
};
console.log(dict["hello"]);   // Hi
```

Allows any string key with string value.

2. keyof Operator

```
interface User {
    id: number;
    name: string;
    email: string;
}
type UserKeys = keyof User;   // "id" | "name" | "email"
let key: UserKeys = "name";   // only valid keys allowed
```

3. Using keyof with Function

```
function getValue(obj: User, key: keyof User) {
    return obj[key];
}
console.log(getValue({ id: 1, name: "Aman", email: "a@gmail.com" },
                     "email"));    // a@gmail.com
```

17. Interfaces vs Types

→ Both are used to define shapes of objects.

→ interface is extendable and can be merged.

→ type is more flexible and can represent unions, tuples, primitives, etc.

→ Use interface for object shapes and type for unions, tuples, or advanced types.

Comparison

Feature	interface	type
Declaration	Extensible	Not extensible
Merging	Yes	No
Union Types	No	Yes
Tuple Types	No	Yes
Primitives	No	Yes
Best for	Objects, Classes	Primitives, Unions, Tuples, Advanced

Example:

1. Using interface

```
interface User {
    id: number;
    name: string;
}

interface User {
    email: string;      // declaration merging
}

const u: User = { id: 1, name: "Aman", email: "a@gmail.com" };
```

2. Using type

```
type UserType = {
    id: number;
    name: string;
};

// Union type
type Status = "PENDING" | "SUCCESS" | "FAILED";

// Tuple type
type Point = [number, number];
const p: Point = [10, 20];
```

18. Type Aliases & String Literal Types

→ Type aliases give a new name to any type.

→ String literal types allow us to define a specific string value.

→ Useful for restricted values and improving code readability.

String Literal Type Example

```
type Direction = "up" | "down" | "left" | "right";
let move: Direction = "left";    // ☑ valid
// let move2: Direction = "forward"; // ✖ error
```

Pro Tip:

Use string literal types for states, actions, roles, or fixed sets of values.

Example:

1. Type Alias

```
type ID = number | string;
type UserId = number;

type UserInfo = {
    id: UserId;
    name: string;
    email: string;
};
const user: UserInfo = { id: 1, name: "Aman", email: "a@gmail.com" };
```

> Type aliases make complex types easy to reuse and maintain.

2. String Literal Type

```
type Role = "ADMIN" | "EDITOR" | "VIEWER";
let role: Role = "ADMIN";        // ☑ valid
// let role2: Role = "GUEST";    // ✖ error
```

3. Combining Both

```
type ResponseStatus = "SUCCESS" | "FAILED";

type ApiResponse<T> = {
    status: ResponseStatus;
    data: T;
    message: string;
};
```

19. Clusters in TypeScript

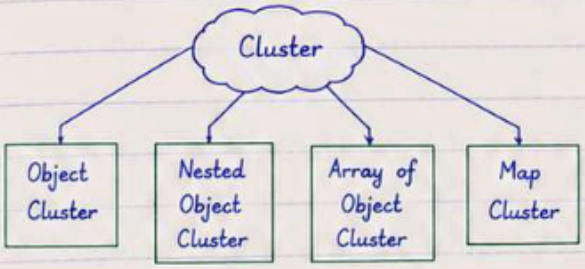
→ Clusters are groups of related data values stored in a structured way.

→ They help us manage complex data together as one unit.

→ In TypeScript, we can create clusters using objects, nested objects, arrays of objects, or maps.

→ Useful for real-world data like user profiles, product info, orders, etc.

Cluster Concept



Types of Clusters

Type	Description	Example
Object Cluster	Group related values using an object.	User with id, name, email
Nested Object Cluster	Object inside another object.	Address inside User object
Array of Object Cluster	Array that stores multiple objects.	List of users or products
Map Cluster	Key-value pairs stored in a Map.	ProductId → Product Object

Pro Tip:

Use clusters to keep your data organized, readable, and easy to maintain.

Avoid using many independent variables.

Examples:

1. Object Cluster

```
const user = {
    id: 1,
    name: "Aman",
    email: "aman@gmail.com",
    isActive: true
};
```

All user related info in one cluster.

2. Nested Object Cluster

```
const user = {
    id: 1,
    name: "Aman",
    address: {
        street: "MG Road",
        city: "Gurgaon",
        pincode: 122001
    }
};
```

Address is a nested cluster inside user.

3. Array of Object Cluster

```
const users = [
  { id: 1, name: "Aman", email: "a@gmail.com" },
  { id: 2, name: "Neha", email: "n@gmail.com" },
  { id: 3, name: "Rohit", email: "r@gmail.com" }
];
```

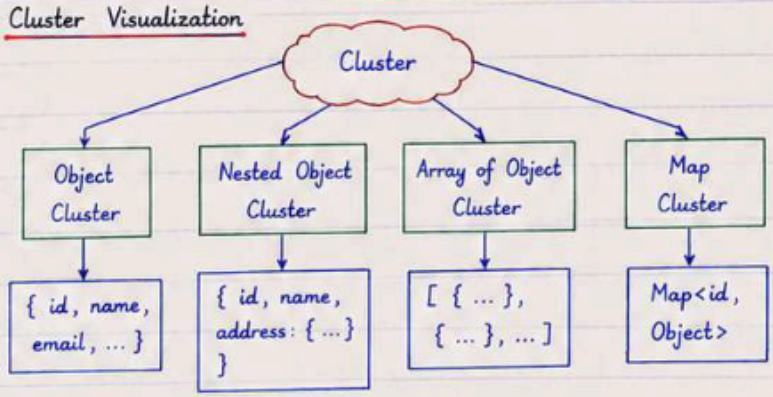
An array containing multiple user clusters.

4. Map Cluster

```
const productMap = new Map<number, {
    name: string, price: number
}>();
productMap.set(101, { name: "Laptop", price: 45000 });
productMap.set(102, { name: "Mouse", price: 800 });
```

Key-value pairs grouped as a cluster.

Cluster Visualization



Summary:

→ Clusters help us group related data.

→ Use objects, nested objects, arrays, or maps.

→ Makes code clean, scalable, and maintainable.